

طراحی و پیاده‌سازی پشتیبانی از دستورالعمل‌های با طول متغیر در پردازنده RISC-V

۱. مقدمه

استاندارد ۳۲ بیتی و هم دستورالعمل‌های فشرده ۱۶ بیتی را اجرا کند؛ بنابراین مکانیزم واکنشی و کنترل باید بین این دو طول تمایز قائل شود. می‌کند و PC پس از هر دستورالعمل ۴ واحد افزایش می‌یابد. برای پشتیبانی از دستورالعمل‌های فشرده، پردازنده باید هم دستورالعمل‌های پردازنده اصلی فقط دستورالعمل‌های ۳۲ بیتی را اجرا

۲. تغییر حافظه دستورالعمل

۳۲ بیتی به دو دسترسی متوالی نیاز دارد. این کار پهنای حافظه را کاهش می‌دهد و امکان پشتیبانی دستورالعمل فشرده را فراهم می‌کند. در طرح اصلاح‌شده از حافظه‌ای با پهنای ۱۶ بیت استفاده می‌شود؛ در نتیجه یک دستورالعمل فشرده به یک دسترسی و یک دستورالعمل در طراحی اصلی، حافظه دستورالعمل در هر دسترسی ۳۲ بیت خروجی می‌داد.

۳. مکانیزم واکنشی دستورالعمل

۳.۱ واکنشی دستورالعمل ۱۶ بیتی (حالت فشرده)

یک نیم‌واژه از حافظه خوانده و مستقیم به رمزگشا ارسال می‌شود؛ PC به اندازه ۲ بایت افزایش می‌یابد و اجرا در یک چرخه کامل می‌شود.

۳.۲ واکنشی دستورالعمل ۳۲ بیتی (حالت عادی)

واکنشی اول: ۱۶ بیت پایینی خوانده، در رجیستر موقت ذخیره و $PC = PC + 2$ می‌شود؛ سپس بقیه پردازنده متوقف (stall) می‌گردد. بیت بالایی خوانده و با نیمه اول ترکیب می‌شود؛ توقف برداشته، دستورالعمل رمزگشایی و اجرا شده و PC دوباره ۲ واحد افزایش می‌یابد. واکنشی دوم: ۱۶ مجموع افزایش PC در این دو گام برابر با طول اصلی ۴ بایتی دستورالعمل است.

۴. رجیستر موقت دستورالعمل

اول را ذخیره، منتظر نیمه دوم می‌ماند و سپس هر دو نیمه را ترکیب می‌کند. بدون این رجیستر، رمزگشا دستورالعمل ناقص دریافت می‌کرد. یک رجیستر موقت ۱۶ بیتی برای نگه‌داشتن نیمه اول هر دستورالعمل ۳۲ بیتی معرفی شد که نیمه

۵. مکانیزم توقف پردازنده (Stall)

(freeze) می‌شود تا از ورود دستورالعمل ناقص به مراحل اجرا جلوگیری شود. توقف بلافاصله پس از تکمیل دستورالعمل برداشته می‌شود. در حین واکنشی نیمه دوم یک دستورالعمل ۳۲ بیتی، مرحله واکنشی به کار خود ادامه می‌دهد اما بقیه خط‌لوله منجمد

۶. عملکرد شمارنده برنامه (PC)

است؛ تنها مقدار افزایش تغییر می‌کند. چون هر دسترسی حافظه یک نیم‌واژه باز می‌گرداند، واکنشی متوالی همواره به صورت زیر است PC همچنان رجیستری ۳۲ بیتی

$$PC_{next} = PC + 2$$

برای دستورالعمل ۳۲ بیتی: دو واکنشی متوالی، هر کدام $PC + 2$ مجموع $PC + 4$. برای دستورالعمل فشرده: تنها یک واکنشی، $PC + 2$.

۷. حالت‌های اجرایی پردازنده

حالت عادی (Normal): اجرای دستورالعمل ۳۲ بیتی – هر دستورالعمل نیازمند دو دسترسی حافظه.
حالت فشرده (Compressed): اجرای دستورالعمل ۱۶ بیتی – هر دستورالعمل نیازمند یک دسترسی حافظه.

۸. انتقال حالت پردازنده – رجیستر Thumb

حالت عادی (۳۲ بیتی) و $Thumb = 1$ یعنی حالت فشرده (۱۶ بیتی). این رجیستر تنها با اجرای دستورالعمل‌های کنترلی خاص تغییر می‌کند. یک رجیستر یک‌بیتی به نام Thumb حالت اجرایی فعلی را مشخص می‌کند: $Thumb = 0$ یعنی ورود به حالت فشرده: با تشخیص دستورالعمل سفارشی Thumb، $0xFFFF$ از ۰ به ۱ تغییر می‌کند. خروج از حالت فشرده: با تشخیص $Opcode = 110$ یا $Funct3 = 010$ یا $Funct3 = 011$ از ۱ به ۰ باز می‌گردد.

۹. منطق انتقال حالت (D Flip-Flop)

رجیستر Thumb به صورت یک D فلیپ‌فلاپ با معادله حالت بعدی زیر پیاده‌سازی می‌شود:

$$Thumb_{next} = Thumb \oplus Toggle, \quad Toggle = EnterCompressed \vee ExitCompressed$$

دستورالعمل‌های انتقال فعال می‌شود؛ در غیر این صورت $Toggle = 0$ و $Thumb_{next} = Thumb$ ، یعنی پردازنده در حالت فعلی باقی می‌ماند. سیگنال Toggle تنها با رمزگشایی یکی از

۱۱. نتیجه‌گیری

است. این راه‌حل معماری اصلی را حفظ کرده و اجرای کارآمد هر دو قالب دستورالعمل را با حداقل سربار سخت‌افزاری امکان‌پذیر می‌سازد. رجیستر موقت واکنشی، توقف پردازنده در واکنشی چندچرخه‌ای، و رجیستر حالت Thumb برای کنترل انتقال میان اجرای استاندارد و فشرده مکانیزم واکنشی دستورالعمل با طول متغیر با موفقیت در پردازنده RISC-V ادغام شد.